Either & Railway-Oriented Programming

Validate user id pipeline without exceptions
Season 6 · Functional · Intermediate

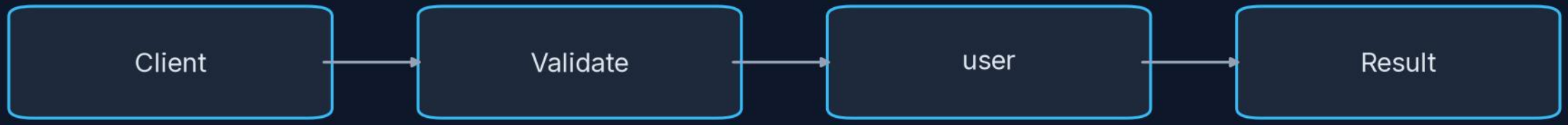
Today's Hook

- Problem: Validate user id pipeline without exceptions
- Naive pain cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- We will refactor to Either/Railway with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Validate user id pipeline without exceptions
- Constraints: solve «Validate user id pipeline without exceptions» with Either/Railway — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Validate user id pipeline without exceptions
- Success looks like: caught

Structure



Either/Railway: Validate user id pipeline without exceptions

Real-World Scenario

- Running example: Validate user id pipeline without exceptions
- Key types: Either, Left, Right, UserPipeline
- Happy-path stdout cue: caught
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.either.problem;

import java.util.*;

public final class NaiveUser {
  public static String load(String id){ if(id==null) throw new RuntimeException("null"); return "user:"+id; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Validate user id pipeline without exceptions
- Without Either/Railway, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.either.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Either<L,R> permits Left, Right { boolean isRight(); L left(); R right(); }
record Left<L,R>(L value) implements Either<L,R> { public boolean isRight(){return false;} public L left(){return value;} public R right(){throw new IllegalStateException();} }
record Right<L,R>(R value) implements Either<L,R> { public boolean isRight(){return true;} public L left(){throw new IllegalStateException();} public R right(){return value;} }
final class UserPipeline {
  static Either<String,Integer> parseInt(String s){ try { return new Right<>(Integer.parseInt(s)); } catch(Exception e){ return new Left<>("parse"); } }
  static Either<String,Integer> requirePositive(int n){ return n>0? new Right<>(n): new Left<>("positive"); }
  static Either<String,String> label(int n){ return new Right<>("user:"+n); }
  static Either<String,String> run(String raw){
    var a=parseInt(raw); if(!a.isRight()) return new Left<>(a.left());
    var b=requirePositive(a.right()); if(!b.isRight()) return new Left<>(b.left());
    return label(b.right());
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.either.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Either: user id railway ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveUser.load(null)); } catch(Exception e){ System.out.println("caught"); }
System.out.println("--- either ---");
System.out.println(UserPipeline.run("42"));
System.out.println(UserPipeline.run("x"));
System.out.println(UserPipeline.run("-1"));
  }
}
```

Walkthrough Output

```
=== Either: user id railway ===
--- naive ---
caught
--- either ---
Right[value=user:42]
Left[value=parse]
Left[value=positive]
```

Step-by-Step Trace

- Match stdout to stages — look for: caught
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 43-either-railway ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.functional.either.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Either<L,R> permits Left, Right { boolean isRight(); L left(); R right(); }
record Left<L,R>(L value) implements Either<L,R> { public boolean isRight(){return false;} public L left(){return value;} public R right(){throw new IllegalStateException();} }
record Right<L,R>(R value) implements Either<L,R> { public boolean isRight(){return true;} public L left(){throw new IllegalStateException();} public R right(){return value;} }
final class UserPipeline {
  static Either<String,Integer> parseInt(String s){ try { return new Right<>(Integer.parseInt(s)); } catch(Exception e){ return new Left<>("parse"); } }
  static Either<String,Integer> requirePositive(int n){ return n>0? new Right<>(n): new Left<>("positive"); }
  static Either<String,String> label(int n){ return new Right<>("user:"+n); }
  static Either<String,String> run(String raw){
    var a=parseInt(raw); if(!a.isRight()) return new Left<>(a.left());
    var b=requirePositive(a.right()); if(!b.is
```

Trade-offs

- Use Either/Railway when «Validate user id pipeline without exceptions» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Either/Railway in product language.

Related Patterns

- Name the pattern usually confused with Either/Railway and separate the forces.
- Sketch the roles for «Validate user id pipeline without exceptions» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Validate user id pipeline without exceptions
- Naive pain → Either/Railway boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.